6/1/2017

EE 241 DIGITAL CIRCUITS
FINAL EXAM SOLUTION MANUAL

Name:

Q1) (20 points) You are asked to design a synchronous frequency divider. Within your module input clock frequency will be divided by **six**. Provide the complete Verilog description of your module.

Bonus: **(10 points)** *Generalize your frequency divider module in Verilog such that the user can enter the division value as 2, 4, 6, 8, 10, 12, and 14.*

SOLUTION:

```
module clock_divider(clk_out,clk);

input clk;
output reg clk_out;

reg [3:0] clkc=0;

parameter divider = 4'd3;

always @ (negedge clk)
begin
clkc <=clkc+1;
clk_out<=clkc / divider;
if (clkc==(2*divider-1)) clkc<=0;
end

endmodule
```

```
`timescale 1ns / 1ps
module clock_divider_tb;

reg clk;

wire clk_out;

// Instantiate the Unit Under Test (UUT)
clock_divider UUT (clk_out,clk);
```

```
initial begin
clk=0;
forever
#3 clk=~clk;
end

endmodule
```

Q2) (30 points) To show a hexadecimal number on four seven-segment display digits of the Basys3 board, we should first convert it to binary coded decimal (BCD) form. As a simple example, hexadecimal number FF corresponds to 255 in decimal form. Its BCD representation will have a hundreds digit as 2, tens digit as 5, and ones digit as 5.

Form a module in Verilog using dataflow model which gets the hexadecimal number (num) as input. Your hexadecimal number can be at most FFF. Output of the module will be thousands digit (thos), hundreds digit (huns), tens digit (tens), and ones digit (ones).

SOLUTION:

```
module test_bcd(num,thos,huns,tens,ones);

input  [15:0] num;
output [3:0] thos;
output [3:0] huns;
output [3:0] tens;
output [3:0] ones;


assign thos=num/16'd1000;
assign huns=(num-thos*16'd1000)/16'd100;
assign tens=(num-thos*16'd1000-huns*16'd100)/16'd10;
assign ones=(num-thos*16'd1000-huns*16'd100-tens*16'd10);
endmodule


module test_bcd_tb;

reg  [15:0] num;
wire [3:0] thos;
wire [3:0] huns;
wire [3:0] tens;
wire [3:0] ones;

test_bcd UUT(num,thos,huns,tens,ones);

initial begin
num = 4126;

#100;
end

endmodule
```

Q3) (20 points) An external source feeds bit values to your system at each clock cycle. Your system should detect the location of ones (relative to each other). These locations cannot be longer than 15. When a one is not detected your output should be zero. As an example, when the sequence 00001000001 comes, your outputs should be
00005000006.

Form a complete Verilog description for your system.

```
module logic_one_detector(location,clk,value);

input clk,value;
output reg [3:0] location;

reg [3:0] clkc=0;

always @ (posedge clk)
begin
clkc <=clkc+1;
if (value==1'b1)
begin
clkc<=0;
location <= clkc + 1;
end
else
location <= 4'h0;
end

endmodule
```

```
`timescale 1ns / 1ps
module logic_one_detector_tb;

reg clk, value;

wire [3:0] location;

// Instantiate the Unit Under Test (UUT)
logic_one_detector UUT (location,clk,value);

initial begin
clk=0;
forever
#2 clk=~clk;
end
```

```
initial
begin
#1 value = 1'b0;
#4 value = 1'b0;
#4 value = 1'b0;
#4 value = 1'b0;
#4 value = 1'b0;
#4 value = 1'b0;
#4 value = 1'b1;//7
#4 value = 1'b0;
#4 value = 1'b0;
#4 value = 1'b0;
#4 value = 1'b0;
#4 value = 1'b0;
#4 value = 1'b0;
#4 value = 1'b0;
#4 value = 1'b1;//8
#4 value = 1'b0;
#4 value = 1'b0;
#4 value = 1'b0;
#4 value = 1'b1;//4
#4 value = 1'b0;
#4 value = 1'b0;

end

endmodule
```

Q4) (30 points) Stepper motor is a device that rotates in steps. The rotation step size can be 1.8 degrees in full-stepping. Therefore, a full rotation needs 200 steps. Speed of the motor is determined by the time delay between each step.

We should feed a binary sequence to rotate the stepper motor. For forward rotation, the sequence will be

First value **1000**
Second value **0100**
Third value **0010**
Fourth value **0001**

We should feed this sequence in a periodic manner to rotate the stepper motor in forward rotation. If we want backward rotation, then the sequence should be fed in reverse order. We will connect the stepper motor to the Basys3 board.

Form Verilog modules to

a- rotate the motor by a given degree.
b- adjust the speed of the motor by adding delays of 0.2, 0.4, 0.6, and 0.8 sec between steps.
c- set the rotation direction as forward or backward.
d- Construct a top module for your stepper motor driver.
- Rotation degree will be taken from switches of the Basys3 board in hexadecimal form.
- Binary sequence to be fed to the stepper motor should be observed on LEDs of the board.
- Rotation direction should be set by a switch on the Basys3 board.
- Rotation speed should be set by switches on the board.

SOLUTION:

```
module top_module(led,sw,clk);

input [11:0] sw;
input clk;

output [3:0] led;

wire [11:0] stepnum;

degreetostep U1(sw[8:0],stepnum);

wire clk_out;

clock_divider U2(clk_out,clk,sw[10:9]);

stepper_sequence U3(led[3:0],stepnum[11:0],sw[11],clk_out);

endmodule
```

```
`timescale 1ns / 1ps

module top_module_tb;

reg clk;
reg [11:0] sw;

wire [3:0] led;

top_module UUT(led,sw,clk);

initial begin
#1 clk=0;
forever
#1 clk=~clk;
end

initial
begin

sw[8:0]=9'b100101100;
sw[10:9]=2'b00;
sw[11]=1'b0;

end
endmodule



module clock_divider(clk_out,clk,div_coef);

input clk;
input [1:0] div_coef;

output reg clk_out;

reg [23:0] clkc=0;

//set for 0.2 sec
parameter divider = 24'd10000000;

always @ (negedge clk)
begin
clkc <=clkc+1;
clk_out<=clkc / (divider*(div_coef+1'b1));
if (clkc==(2*divider*(div_coef+1'b1)-1)) clkc<=0;
end
```

```
endmodule



`timescale 10ns / 1ps
module clock_divider_tb;

reg clk;
reg [1:0] div_coef;

wire clk_out;

// Instantiate the Unit Under Test (UUT)
clock_divider UUT (clk_out,clk,div_coef);

initial begin
clk=0;
div_coef=2'b00;

forever
#1 clk=~clk;
end

endmodule



module degreetostep(degree,stepnum);

input [8:0] degree;
output [11:0] stepnum;

assign stepnum = ((degree%9'd360)*4'd10)/5'd18;

endmodule



`timescale 10ns / 1ps
module degreetostep_tb;

reg [8:0] degree;

wire [11:0] stepnum;

// Instantiate the Unit Under Test (UUT)
degreetostep UUT(degree,stepnum);

initial begin

degree=9'd220;
```

```
end

endmodule



module stepper_sequence(sequence,stepnum,direction,clk);

input [7:0] stepnum;
input clk,direction;

output reg [3:0] sequence = A;

reg [7:0] stepcount=8'h00;

parameter A=4'b1000, B=4'b0100, C=4'b0010, D=4'b0001;

always @ (posedge clk)
begin
if (stepcount < stepnum)
begin
   if (direction == 0)
   case (sequence)
   A : sequence <= B;
   B : sequence <= C;
   C : sequence <= D;
   D : sequence <= A;
   endcase
   else
   case (sequence)
   A : sequence <= D;
   B : sequence <= A;
   C : sequence <= B;
   D : sequence <= C;
   endcase
   stepcount <= stepcount+1;
   end
else stepcount <= stepnum;
end
endmodule
```

```
`timescale 1ns / 1ps

module stepper_sequence_tb;

reg clk,direction;
reg [7:0] stepnum;

wire [3:0] sequence;

stepper_sequence UUT(sequence,stepnum,direction,clk);

initial begin
#1 clk=0;
forever
#1 clk=~clk;
end

initial
begin

direction = 1'b0;
stepnum = 8'd10;

end
endmodule
```